
Security of Computer Systems

Project Report

Authors:

Alicja Sobiech, 188861

Miłosz Gonsior, 193475

Version: 2.5

Versions

Versions	Date	Description of changes
1.0	04.04.2026	Creation of document
1.1	07.04.2026	Filling out the control meeting section
2.0	17.05.2026	Translating the document from .docx to .tex
2.1	31.05.2026	Initial version of section 2
2.2	07.06.2026	Filling out section 2
2.3	08.06.2026	Completing the bibliography
2.4	08.06.2026	Completing section 2
2.5	08.06.2026	Final formatting improvements

1. Project - control term

1.1. Description

The aim of the overall project is the implementation of a set of applications emulating an environment with trusted third party (TTP), server and a client.

1.2. Results - implemented functionalities

- Docker configuration
- Basic version of server and TTP applications
- RSA key pair generation

1.3. Plans for future work

- Design and implementation of a Graphical User Interface (GUI)
 - Icons presenting the state of the application
- Implementation of the Client
- Running tests (eg. sample attacks)
- Generation of documentation using Doxygen

2. Project - Final term

2.1. Achieved results

The result of our work is the developement of 3 python applications emulating a user/client, a server and a trusted third part (TTP) according to the project instruction ^[1]. The network enviroment is created using Docker containers. The cryptographic tasks are done using the cryptography package.

The main application is the client service and it's GUI. It allows for authentication cia the trusted third party (TTP), obtaining an AES key and sending encrypted messages to the server application.

The TTP and Server application work in the background in relation to the Client service. The main tasks of the TTP service is generating certificates and facilitating authorisation attempts. The Server service receives and decrypts messages from the user.

2.2. Key functionalities

2.2.1. Key handling

4096-bit keys are generated as a private, public pair using `generate_private_key()` and `public_key()` methods respectively.

AES key is generated using `urandom()` method.

```
def generate_RSA_key_pair():
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=4096)
    public_key = private_key.public_key()
    return public_key, private_key

def generate_aes_key():
    return os.urandom(32)
```

List.1 RSA and AES key generation

The key serialization converts them into PEM-encoded bytes. The public key is encrypted during this process while the private key remains without encryption. Deserialization is done using `load_pem_public_key()` and `load_pem_private_key()` methods.

```
def serialize_public_key(public_key):
    return public_key.public_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PublicFormat.SubjectPublicKeyInfo)

def deserialize_public_key(serialized_public_key):
    return serialization.load_pem_public_key(serialized_public_key)
```

List.2 Public key serialization and deserialization

```

def serialize_private_key(private_key):
    return private_key.private_bytes(
        encoding=serialization.Encoding.PEM,
        format=serialization.PrivateFormat.PKCS8,
        encryption_algorithm=serialization.NoEncryption())

def deserialize_private_key(serialized_private_key):
    return serialization.load_pem_private_key(
        serialized_private_key,
        password=None)

```

List.3 Private key serialization and deserialization

2.2.2. Certificate handling

The certificate is generated using `x509.CertificateBuilder()` using the given public key of the certificate subject and signed using the issuer's private key. Certificate generated in that way is valid for 4 weeks from the moment of generation.

```

def generate_certificate(subject_id, subject_public_key,
                        issuer_id, issuer_private_key):
    builder = x509.CertificateBuilder()

    builder = builder.subject_name(x509.Name(
        [x509.NameAttribute(NameOID.COMMON_NAME, subject_id)]))
    builder = builder.issuer_name(x509.Name(
        [x509.NameAttribute(NameOID.COMMON_NAME, issuer_id)]))
    builder = builder.public_key(subject_public_key)
    builder = builder.serial_number(x509.random_serial_number())
    builder = builder.not_valid_before(datetime.datetime.now())
    builder = builder.not_valid_after(
        datetime.datetime.now() + datetime.timedelta(weeks=4))

    return builder.sign(issuer_private_key, hashes.SHA256())

```

List.4 Certificate generation

A certificate generated in the way described above can also be serialized and later deserialized. These processes are done with `cert.public_bytes()` and `x509.load_pem_x509_certificate()` methods respectively.

```

def serialize_certificate(cert):
    return cert.public_bytes(serialization.Encoding.PEM)

def deserialize_certificate(serialized_cert):
    return x509.load_pem_x509_certificate(serialized_cert)

```

List.5 Certificate serialization and deserialization

2.2.3. Data encryption and decryption

Data can be encrypted and decrypted in two ways: using RSA (Rivest-Shamir-Adleman algorithm) with OAEP padding and SHA-256 algorithm or using AES (Advanced Encryption Standard) with CFB mode (Cipher Feedback Mode).

```
def encrypt_data(public_key, data):
    return public_key.encrypt(
        data,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )

def decrypt_data(private_key, encrypted_data):
    return private_key.decrypt(
        encrypted_data,
        padding.OAEP(
            mgf=padding.MGF1(algorithm=hashes.SHA256()),
            algorithm=hashes.SHA256(),
            label=None
        )
    )
```

List.6 Data encryption and decryption using RSA-OAEP with SHA-256

```
def encrypt_aes(key, plaintext):
    iv = os.urandom(16)
    cipher = Cipher(algorithms.AES(key), modes.CFB(iv))
    encryptor = cipher.encryptor()
    ciphertext = iv + encryptor.update(plaintext) + encryptor.finalize()
    return ciphertext

def decrypt_aes(key, ciphertext):
    iv = ciphertext[:16]
    cipher = Cipher(algorithms.AES(key), modes.CFB(iv))
    decryptor = cipher.decryptor()
    plaintext = decryptor.update(ciphertext[16:]) + decryptor.finalize()
    return plaintext
```

List.7 Data encryption and decryption using AES

2.3. Regular flow test

Regular flow test of our system is conducted by realizing the following steps:

1. Click the "Log in with TTP" button
2. Click the "Download AES key" button
3. Type a message into the textbox
4. Click the "Send message" button

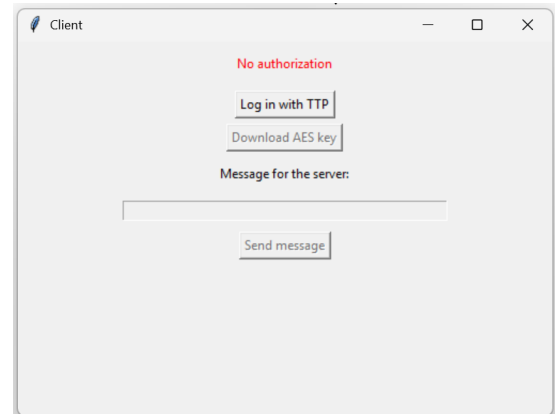


Fig.1 Client app before flow test

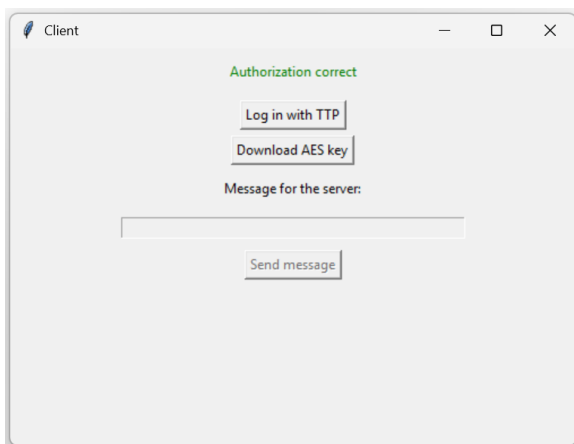


Fig.2 GUI state after step 1

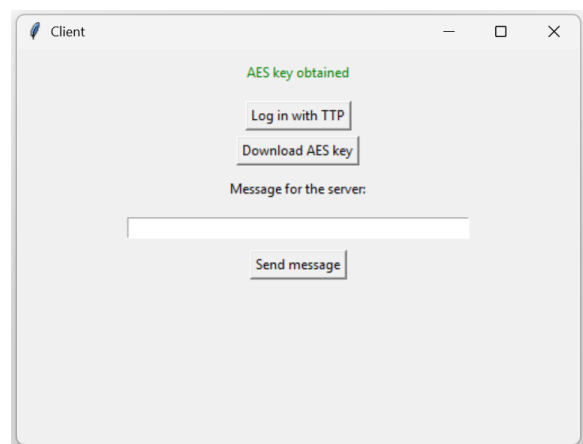


Fig.3 GUI state after step 2

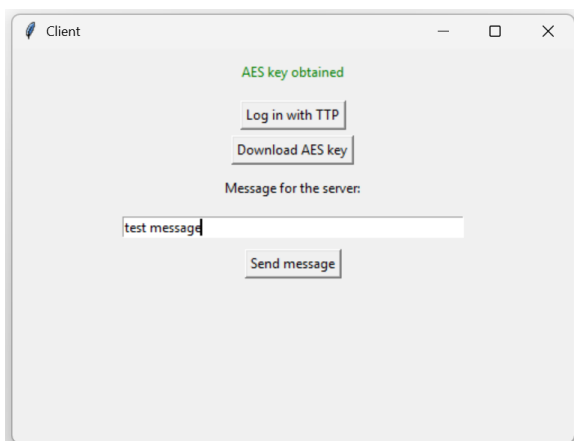


Fig.4 GUI state after step 3

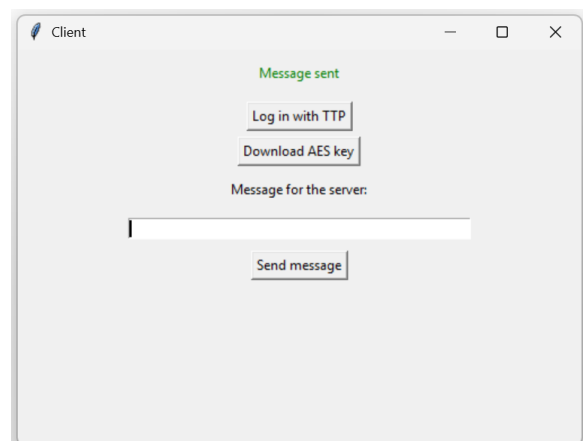


Fig.5 GUI state after step 4

```
Received AES key: 93f6e958b21531e213bdfbcac79b420160ff8cccf13deec8edff5f7cb2cf60bb
Encrypted message: 87b9dbb12f6d7c03a6257d4135ed8174000383d15e0a370af11881d8
```

Fig.6 Client app console after step 4

```
bsk_server | 2026-06-08 10:10:55,404 - Server_App - INFO - Decrypted data from client: test message
```

Fig.7 Decrypted message in server console after step 4

2.4. Performed vulnerability tests and their results

2.4.1. Mismatch of client IDs between TTP and the server

In order to test how our applications behave when client IDs sent to the server and TTP are different we temporarily modified our code, so that TTP received the correct client ID, and the server received a fake ID.

As a result we were not able to obtain the AES key.

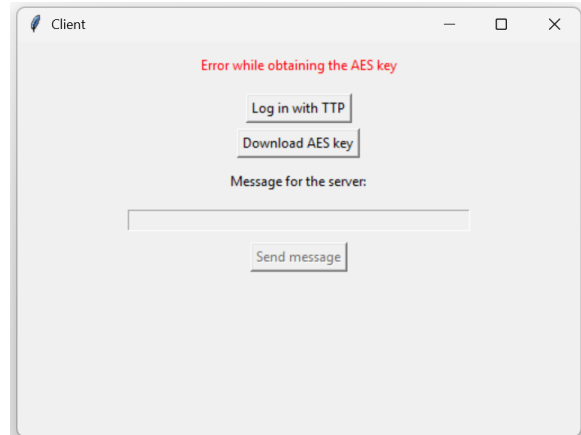


Fig.8 GUI state after attempting to obtain an AES key

```
client_id = crypto.generate_random_id()

s.connect((server_host, server_port))
payload = { "type": "service_request", "client_id": "fake ID" }

ttp_s.connect((ttp_host, ttp_port))
payload={ "type": "fetch_key", "client_id": client_id }
```

List.8 Payloads for server (s) and ttp (ttp_s) simulating mismatch in client IDs

2.4.2. Forged certificate

??? do we even have certificate checking

2.5. Summary

Literature

- [1] Rajchowski P.: *Security of Computer Systems - Project. Emulating environment with Trusted Third Party and Client-Server data exchange scenario* [Project instruction], Gdańsk (Gdańsk University of Technology), 23.02.2026
- [2] Online *Doxygen* documentation, [accessed on 07.06.2026], Available at: <https://www.doxygen.nl/manual/lists.html>
- [3] Online *cryptography* documentation, [accessed on 07.06.2026], Available at: <https://cryptography.io/en/latest/>
- [4] Mumba E.: *Mastering Python Function Documentation: A Practical Guide*, In: DeepDocs, 15.11.2025 [accessed on 07.06.2026], Available at: <https://deepdocs.dev/function-documentation-python/>
- [5] *Privacy-Enhanced Mail*, In: Wikipedia, [online], 27.02.2026, [accessed on 06.06.2026], Available at: https://en.wikipedia.org/wiki/Privacy-Enhanced_Mail
- [6] *X.509*, In: Wikipedia, [online], 22.04.2026, [accessed on 07.06.2026], Available at: <https://en.wikipedia.org/wiki/X.509>
- [7] *SHA-2*, In: Wikipedia, [online], 24.05.2026, [accessed on 07.06.2026], Available at: <https://en.wikipedia.org/wiki/SHA-2>
- [8] Moriarty K. et al.: *PKCS #1: RSA Cryptography Specifications Version 2.2*, In: RFC Editor, United States, 01.11.2016, 8017, ISSN: 2070-1721, Available at: <https://doi.org/10.17487/RFC8017>
- [9] *RSA Algorithm in Cryptography*, In: GeeksForGeeks, 23.07.2025, [accessed on 07.06.2026], Available at: <https://www.geeksforgeeks.org/computer-networks/rsa-algorithm-cryptography/>
- [10] Abhinand S.: *Advanced Encryption Standard (AES)*, In: GeeksForGeeks, 08.08.2025, [accessed on 07.06.2026], Available at: <https://www.geeksforgeeks.org/computer-networks/advanced-encryption-standard-aes/>
- [11] Kattamuri M.: *Block Cipher modes of Operation*, In: GeeksForGeeks, 30.05.2026, [accessed on 07.06.2026], Available at: <https://www.geeksforgeeks.org/ethical-hacking/block-cipher-modes-of-operation/>